

多 Agent 自进化量化投研系统架构方案

目标受众：OpenClaude / Claude Code / Kimi Code 等 AI 编程助手

项目目标：构建一套 7×24 小时运行、自我迭代、覆盖 Polymarket（预测市场）与中国期货市场的自动化投研与交易系统。

1. 系统总体架构设计

本系统采用多 Agent 协作架构（Multi-Agent System），将复杂的投研与交易流程拆解为多个职责单一、相互监督的 Agent。系统核心在于“自进化闭环”：从数据收集 -> 策略生成 -> 回测验证 -> 模拟/实盘执行 -> 结果复盘 -> 策略优化的完整循环。

1.1 核心 Agent 角色定义

Agent 角色	核心职责	依赖工具/数据源	关键输出
Data Agent (数据收集者)	7×24 监控市场异动、抓取新闻、社交媒体情绪、宏观数据及量价数据。	Brave Search, 国际互联网, 交易所 API (Polymarket/期货)	结构化市场特征数据、情绪指数、异动警报
Research Agent (策略研究员)	基于 Data Agent 提供的数据，结合 Alpha 派 skill，挖掘市场失效逻辑，提出交易假设。	Spark (逻辑推理), Alpha 派 skill, 历史数据库	策略逻辑文档 (自然语言描述)、因子公式
Coder Agent (策略编写者)	将 Research Agent 的自然语言逻辑转化为可执行的代码（如 Python/Pine Script），并编写回测脚本。	Kimi Code / OpenClaude, 本地 Python 环境	策略源代码、回测框架集成代码
Backtest Agent (回测验证者)	在沙盒环境中运行 Coder Agent 编写的策略，进行样本内/外测试，输出绩效指标。	本地回测引擎 (如 Backtrader/vn.py), 历史行情数据	回测报告 (夏普比率、最大回撤、胜率等)
Risk Agent (风控官)	独立于策略生成链路，审查策略逻辑和回测结果，设定仓位限制和熔断机制。拥有一票否决权。	预设风控规则库, 实时账户权益数据	策略准入许可、动态仓位建议、熔断指令

Execution Agent (交易执行者)	接收通过风控的交易信号，通过 API 路由到对应市场（Polymarket 或期货）执行下单。	交易所交易 API, 订单路由算法	交易执行日志、滑点分析
Evolution Agent (进化复盘者)	对比策略的“预期回测表现”与“实际运行表现”，分析归因，将经验反馈给 Research 和 Coder Agent。	交易执行日志, 回测报告, Spark (深度反思)	策略迭代建议、废弃指令、新因子方向

1.2 数据流与进化闭环

1. 感知层：Data Agent 持续获取外部信息。
2. 认知层：Research Agent 提出假设 -> Coder Agent 编写代码 -> Backtest Agent 验证假设。
3. 决策层：Risk Agent 审核通过后，进入模拟/实盘池。
4. 执行层：Execution Agent 负责具体下单。
5. 进化层：Evolution Agent 闭环反馈，形成“提出假设 -> 验证 -> 执行 -> 反思 -> 修正假设”的强化学习过程。

2. 针对特定市场的适配方案

2.1 Polymarket (预测市场) 专区

- 市场特点：事件驱动、二元期权性质、信息敏感度极高。
- Agent 侧重点：
 - Data Agent：需重度依赖 Brave 和国际互联网，高频抓取 Twitter (X)、新闻网站、民调数据（如涉及政治事件）。
 - Research Agent：侧重于 NLP 情感分析和事件概率推演，而非传统的量价技术分析。
 - Execution Agent：需处理 Web3 钱包签名和 Polygon 链上交互。

2.2 中国期货市场专区

- 市场特点：高杠杆、受宏观政策及产业链基本面影响大、量价规律相对成熟。
- Agent 侧重点：
 - Data Agent：需接入稳定的行情源（如 CTP 接口或第三方数据商），抓取宏观经济数据、库存数据、基差数据。

- **Research Agent:** 结合 Alpha 派 skill，侧重于跨期套利、跨品种套利、CTA 趋势跟踪及基本面量化。
- **Execution Agent:** 需对接 CTP 接口（如通过 vn.py），处理极低延迟的订单发送和撤单逻辑。

3. 核心进化机制 (Self-Evolution)

系统不能只是随机生成代码，必须具备“记忆”和“反思”能力：

1. **策略基因库 (Strategy Prompt Library):** Research Agent 维护一个成功策略的“基因库”。当新策略表现良好时，提取其核心逻辑（如“动量+低波动”）存入基因库；表现差的逻辑打上负面标签。
2. **归因分析 (Attribution Analysis):** Evolution Agent 必须回答：“这次亏损是因为逻辑错误、代码 Bug、还是市场环境突变？”
3. **动态权重分配:** 系统同时运行多个策略（策略池），Evolution Agent 根据近期表现动态调整各策略的资金分配权重（表现好的加仓，连续回撤的降权甚至下线）。

4. 风控与熔断 (保命机制)

Risk Agent 是整个系统的最高权限节点，其规则硬编码，不可被其他 Agent 修改。

1. **单日最大亏损熔断:** 账户总权益单日回撤超过 X%，强制平仓所有头寸，暂停 Execution Agent 运行 24 小时。
2. **单策略回撤限制:** 单一策略触及历史最大回撤的 1.5 倍，立即下线该策略。
3. **黑天鹅检测:** Data Agent 监测到 VIX 飙升或重大突发新闻时，Risk Agent 自动降低全系统杠杆率。
4. **代码安全沙盒:** Coder Agent 生成的代码必须在隔离的 Docker 容器中运行回测，防止恶意代码或死循环拖垮系统。

5. 落地实施路线图 (Roadmap)

建议分三个阶段让 OpenClaude 帮你搭建：

- **Phase 1: 纸面富贵 (Paper Trading & Backtesting)**
 - 目标：跑通 Data -> Research -> Coder -> Backtest 的闭环。
 - 动作：不接实盘 API，只用历史数据生成策略并回测。建立策略基因库。
- **Phase 2: 模拟盘试运行 (Forward Testing)**

- 目标：引入 Risk Agent 和 Evolution Agent。
- 动作：接入实时行情，策略产生信号但不真下单（或在测试网下单），记录预期收益与实际行情的偏差（滑点、延迟）。
- **Phase 3: 小资金实盘 (Live Trading)**
 - 目标：全系统闭环运行。
 - 动作：接入 Execution Agent，分配极小资金（如 100 U 或 1 手农产品期货）跑通全流程。

6. 启动 Prompt (直接发给 OpenClaude / Claude Code)

请复制以下内容发送给你的 AI 编程助手：

Markdown

角色设定

你现在是“自进化量化投研系统”的首席架构师兼核心开发 (Lead Architect & Core Developer)。我

任务目标

我们需要从零开始，基于 Python 构建一套多 Agent 协作的量化交易系统。目标市场是 Polymarket 和

架构参考

请参考以下多 Agent 架构进行设计与开发：

1. Data Agent (数据获取)
2. Research Agent (逻辑生成)
3. Coder Agent (代码编写)
4. Backtest Agent (回测验证)
5. Risk Agent (风控熔断 - 最高优先级)
6. Execution Agent (交易执行)
7. Evolution Agent (复盘进化)

当前任务 (Phase 1 启动)

请不要急于写交易接口代码。我们先从最核心的“策略生成与回测闭环”开始。

请为我编写以下基础框架代码：

1. 定义 Agent 的基础类 (BaseAgent)，包含输入、输出和通信接口。
2. 实现一个简单的 `ResearchAgent`，它可以接收一段市场描述 (Mock 数据)，并输出一段策略逻辑。
3. 实现一个 `CoderAgent`，它可以接收 `ResearchAgent` 的输出，并调用 LLM API (请预留接口)。
4. 提供一个主控脚本 (main.py)，将这几个 Agent 串联起来运行。

请确保代码结构清晰、模块化，并充分考虑后续接入真实数据和真实 LLM API 的扩展性。